

MonoPLC: Monoid Algebra as Architectural Constraints for Integrating Non-Real-Time Inputs in IEC 61131-3 PLC Control Systems

Xiaoxiao Zhang

Abstract

Integrating non-real-time sources such as IoT devices and AI services into IEC 61131-3 PLC control loops is a growing challenge in Industry 4.0. Current approaches lack algebraic guarantees on how inputs from multiple sources are composed at runtime. This paper presents MonoPLC, which enforces Monoid axioms (closure, associativity, identity) as architectural constraints. The architecture separates Core Process Control from Monitoring and Optimization inputs using an Effect Monoid, a Product Monoid for independent domain composition, and a Monoid Homomorphism for cross-layer consistency. Validation on TwinCAT through mathematical analysis, property-based testing, and a case study confirms that the axioms hold within IEC 61131-3, new sources require zero modification to core logic, source disconnection is handled without per-source conditional logic, and cross-layer consistency is preserved.

Keywords

Monoid algebra; IEC 61131-3; PLC architecture; algebraic composition; effect system; M+O integration

Section 1: Introduction

Programmable Logic Controllers are the standard execution platform for industrial process control. They operate real-time programs, with cyclic scan times typically in the millisecond range. In recent years, Industry 4.0 and Industry 5.0 developments have driven growing interest in control system interaction with non-real-time external sources such as IoT devices, AI services, and operator panels [1]. These sources have different, typically lower, reliability assumptions than the PLC control loop. The NAMUR Open Architecture (NOA) [6] captures this separation as two domains: Core Process Control (CPC) and Monitoring & Optimization (M+O). CPC must remain deterministic. M+O represents all non-real-time external inputs. Safely integrating M+O inputs into CPC control loops without requiring case-by-case engineering for each new source is a challenge.

The Generalized Device [3] and model-based automation patterns [4] are examples of object-oriented design patterns for PLC software that help organize control logic and hardware

interfaces at the code level. These patterns improve modularity and reuse, though safe composition of components — not the focus of their study — is not addressed. IEC 61499 distributes the system by using event-driven composition of Function Blocks [5]. It focuses on communication between control nodes, not on communication between non-real-time external sources and the control loop. The Asset Administration Shell (AAS) [21] standardises digital representations of industrial assets but does not define algebraic composition rules for the data it carries. Recent work on integrating Large Language Models with industrial control systems [13-15] shows growing interest in connecting intelligent services to PLC systems. The NOA architecture does include a Verification of Request (VoR) concept — a gateway that validates M+O commands before they enter CPC — but the question of how multiple M+O inputs are composed within a single PLC runtime, with what guarantees, remains open.

The Monoid — an algebraic structure equipped with an associative binary operation and an identity element — has proven effective in contexts ranging from distributed data aggregation (MapReduce [19]) to convergent replicated state (CRDTs [20]). The theoretical background is well established [7-11] and reviewed in Section 2.4, where we also discuss how MonoPLC relates to these precedents.

Algebraic composition methods remain rare in the PLC domain. Šusta [17] applied Monoid axioms to model checking of PLC programs, though his approach operates entirely offline and is limited to static analysis. A detailed review appears in Section 2.5. To the best of our knowledge, no work has applied algebraic methods to the runtime integration of non-real-time inputs in IEC 61131-3 PLC systems.

We call this architecture MonoPLC (Monoid + PLC). This paper shows that adopting Monoid axioms (closure, associativity, identity) as architectural constraints, combined with specific design decisions on how external inputs are represented as algebraic elements, provides fail-passive behaviour, extensibility independent of input source type, and cross-layer consistency. These are structural guarantees that arise from the interaction of algebraic laws and design choices, validated through mathematical analysis and automated testing. The algebraic guarantees apply to input composition; the internal control logic is deterministic sequential code validated through testing. This work investigates the following research questions:

- **RQ-a (Feasibility):** Can the Monoid axioms be implemented within IEC 61131-3 and achieve complete separation of CPC from M+O?
- **RQ-b (Extensibility):** Can new input sources be added without modifying core control logic (horizontal extensibility), and can new monitoring domains be added while preserving all existing control logic (vertical extensibility)?
- **RQ-c (Fail-passive behaviour):** Does the Identity axiom, combined with the design decision to represent absent sources as the identity element, guarantee that the system continues to operate correctly when input sources disconnect or produce no output? (Fail-passive means the system continues in its last known state rather than transitioning to a designated safe state.)

- **RQ-d (Cross-layer consistency):** Can the Monoid Homomorphism provide verifiable cross-layer consistency between the PLC and the middleware through a self-verifying check?

We implement the MonoPLC architecture on TwinCAT with a Python middleware layer. We verify its algebraic properties through mathematical analysis and property-based testing. We validate engineering feasibility through a case study with HMI buttons and an LLM agent as M+O input sources, demonstrate extensibility by adding an MQTT source and a pressure control domain, and evaluate robustness through source disconnection and high-frequency stress tests with scan cycle timing measurements.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 introduces the algebraic background. Section 4 describes the research methodology. Section 5 presents the MonoPLC architecture, including a design space of alternative Monoid implementations with different trade-offs. Section 6 provides formal analysis addressing RQ-a through RQ-d. Section 7 validates the approach through a case study. Section 8 discusses the results. Section 9 concludes with directions for future work.

Section 2: Related Work

2.1 PLC Software Architecture and Modularity

PLC software has grown from simple relay-replacement logic to complex systems with thousands of variables and multiple interacting subsystems [2]. Sehr et al. [1] identified limitations of IEC 61131-3, including implicit assumptions on device behaviour, lack of encapsulation, and undisciplined memory sharing that can lead to nondeterminism. Vogel-Heuser et al. [2] surveyed 16 German companies and found that software engineering for automated production systems is lagging behind classical software engineering, with modularity rated as a key challenge.

2.2 Structural Approaches

Several attempts have been made to bring structured software engineering to PLC programming. Faldella et al. [3] proposed the Generalized Device pattern, a multi-layer object-oriented architecture that separates control policies from actuator and sensor mechanisms. Bonfè et al. [4] developed design patterns for model-based automation software, demonstrating reusable patterns from industrial case studies in packaging. Both works use object-oriented principles — state machines, encapsulation, reuse — to improve the modularity of PLC control software within IEC 61131-3, though their focus remains on structuring individual components rather than addressing properties of composed systems.

IEC 61499 [5] addresses a different problem: distributed automation. It defines event-driven Function Blocks that can be distributed across multiple devices and communicate through event connections. IEC 61499 has known semantic ambiguities that have led to different execution behaviours across runtime implementations [5]. The integration of non-real-time external

sources into the control loop falls outside the scope of the standard.

2.3 Integration Requirements

The need for integrating M+O inputs into PLC systems has been recognised at the standardisation level. The NAMUR NOA architecture [6] separates CPC from M+O and defines a Verification of Request (VoR) gateway concept for M+O commands entering CPC. These recommendations establish the domain separation and the need for verification, which MonoPLC assumes. What remains unaddressed is the composition semantics: once M+O inputs have passed the domain boundary, how are they combined within the PLC runtime, and with what algebraic guarantees?

The Asset Administration Shell (AAS), standardised in IEC 63278-1, provides a technology-neutral digital representation of industrial assets through modular Submodels [21]. Each Submodel describes a specific aspect of the asset such as identification, operational data, or condition monitoring. The AAS metamodel can be mapped to multiple technologies including OPC UA (OPC 30270), XML, and JSON. It is designed for interoperability across vendors and lifecycle stages. The AAS defines a standardised information model but does not prescribe how data is composed or what algebraic properties the composition should satisfy.

Recent work has applied Large Language Models to industrial control systems. Xia et al. [13] used LLM agents to enhance flexible modular production, Gill et al. [14] combined LLM agents with digital twins for fault handling, and Xia et al. [15] demonstrated direct LLM-based control of automation systems. These studies focus on what intelligent services can do once connected to PLC systems; how their inputs are composed with those from other sources at runtime is not addressed.

2.4 Algebraic Methods in Software Engineering

The Monoid is among the simplest algebraic structures used in software engineering. The broader theoretical context is well established: Moggi [7] proposed monads as an abstraction for computational effects, Wadler [8] applied monads to programming language design, and subsequent work by Plotkin and Power [10], Plotkin and Pretnar [11], and Rivas and Jaskielioff [16] developed algebraic treatments of effects with increasing sophistication. Orchard and Petricek [9] showed that effect information carries monoid structure. These works form the theoretical background for using algebraic structures to manage composition in software.

MonoPLC applies the Monoid — the simplest structure in this family — as a composition pattern for PLC input integration. The full power of monads, Kleisli structures, or algebraic effect handlers is not required for this application and is not claimed. The Monoid axioms (closure, associativity, identity) alone provide the architectural constraints needed to compose non-real-time inputs with guaranteed properties.

In engineering practice, Monoid properties are exploited for different purposes in different contexts. MapReduce [19] uses associativity to enable parallel data aggregation across distributed nodes. Conflict-free Replicated Data Types [20] use

monoidal merge for convergent state in distributed replicas. MonoPLC uses the same algebraic properties for a different purpose: composition of non-real-time inputs in a single-node PLC. Recent work extends these ideas further: Hou et al. [24] showed that stream programs factor through monoid homomorphisms, and Power [25] applied algebraic approaches to ensuring correctness across system boundaries in distributed data systems.

2.5 Algebraic Methods in the PLC Domain

Few works have connected algebraic composition methods with PLC software. Šusta [17] proved that the composition of PLC instruction semantics, represented as transfer sets, satisfies Monoid axioms, and applied this to support model checking of PLC programs. His work operates entirely offline and is limited to static analysis. Bohlender and Kowalewski [12] used Constrained Horn Clauses for compositional verification of PLC software, decomposing Function Blocks into modes for model checking — an approach that does not employ algebraic composition. Teatro [18] applied category theory to control systems software in C++17, modeling control programs as Moore machines using algebra and coalgebra. This work targets general-purpose control software, not PLC systems or M+O input integration. None of these works applies algebraic composition methods to the runtime integration of M+O inputs in IEC 61131-3 PLC systems.

Section 3: Background — Monoid Algebra

This section introduces the algebraic concepts used in MonoPLC. For each concept, we briefly explain why it is needed for integrating non-real-time inputs into PLC control loops.

3.1 Monoid

A Monoid $(S, \oplus, \varepsilon)$ consists of a set S , a binary operation $\oplus : S \times S \rightarrow S$, and an identity element $\varepsilon \in S$. The following axioms must hold:

- **Closure:** For all $a, b \in S$, $a \oplus b \in S$.
- **Associativity:** For all $a, b, c \in S$, $(a \oplus b) \oplus c = a \oplus (b \oplus c)$.
- **Identity:** For all $a \in S$, $\varepsilon \oplus a = a = a \oplus \varepsilon$.

Examples include integers under addition $(\mathbb{Z}, +, 0)$ and lists under concatenation $([a], ++, [])$.

In MonoPLC, all effects in the system — setpoint changes, start/stop commands, valve control signals, sensor publications — are represented as elements of a single Effect type (a Structured Text STRUCT containing an effect type enumeration, a target identifier, and a numeric value). This Effect type, together with array concatenation as the binary operation and the empty array as the identity element, forms a Monoid. The identity element represents the absence of any effect, a design choice whose architectural consequences are discussed in Section 6.3.

3.2 Product Monoid

Given two Monoids $(S_1, \oplus_1, \varepsilon_1)$ and $(S_2, \oplus_2, \varepsilon_2)$, the Product Monoid is defined on the Cartesian product $S_1 \times S_2$ with component-wise operation:

$$(a_1, a_2) \oplus (b_1, b_2) = (a_1 \oplus_1 b_1, a_2 \oplus_2 b_2)$$

and identity $(\varepsilon_1, \varepsilon_2)$.

It is straightforward to verify that closure, associativity, and identity hold. This generalises to any finite number of components.

A PLC system may monitor multiple domains: temperature control, humidity control, alarm counts, and so on. In MonoPLC, each domain is modelled as a separate Monoid with its own combine operation and identity. They are combined into a single Product Monoid. Adding a new monitoring domain means adding a new component to the product. Existing components are not affected.

3.3 Monoid Homomorphism

A mapping $\varphi : M_1 \rightarrow M_2$ between Monoids $(M_1, \oplus_1, \varepsilon_1)$ and $(M_2, \oplus_2, \varepsilon_2)$ is a Monoid Homomorphism [22] if it preserves composition and identity:

- $\varphi(a \oplus_1 b) = \varphi(a) \oplus_2 \varphi(b)$
- $\varphi(\varepsilon_1) = \varepsilon_2$

In MonoPLC, the PLC side and the middleware side use different Monoids (array concatenation and dictionary merge, respectively). The homomorphism ensures that these two sides combine data consistently regardless of how effects are batched across the communication boundary. The architecture and verification of this mapping are described in Sections 5.5 and 6.4.

Section 4: Research Methodology

4.1 Research Approach

This work follows a three-stage approach. We develop an architecture based on Monoid axioms within IEC 61131-3. We validate its algebraic properties through mathematical analysis and automated testing. We demonstrate engineering feasibility through a case study.

The MonoPLC architecture is designed and implemented on a TwinCAT PLC with a Python middleware layer. Its algebraic properties — that the implemented data structures and operations satisfy the Monoid axioms — are then established mathematically and confirmed through automated testing. Engineering feasibility is demonstrated through a case study with two M+O input sources (HMI buttons and an LLM agent), with extensibility validated by adding an MQTT source and a pressure control domain.

4.2 Why Monoid

We chose the Monoid because it is the simplest algebraic structure that provides composition with guaranteed properties. It requires only a binary operation and an identity element. Both can be implemented in Structured Text using fixed-size arrays and standard functions. The axioms (closure, associativity, identity) are testable. The structure is minimal, but the constraints it imposes produce useful architectural properties as shown in Section 6.

The implementation techniques used in MonoPLC — array concatenation, dictionary merge — are standard operations familiar to any PLC or middleware programmer. Recognising algebraic structure in such operations is a well-established route to algorithmic guarantees (Section 2.4). The contribution is not in the operations themselves, but in identifying that they satisfy Monoid axioms and enforcing these axioms as architectural constraints. Algebraic structure, once made explicit, enables compositional reasoning, local re-verification under change, and principled default behaviour. Whether these benefits hold when Monoid axioms are applied to PLC input composition is what this paper sets out to investigate.

4.3 Validation Strategy

Algebraic properties (associativity, identity, homomorphism preservation) are established through mathematical analysis and confirmed through property-based testing [23] using the Hypothesis framework [26], in which universally quantified properties are checked against randomly generated inputs. Engineering feasibility and extensibility are validated through a case study: we add new M+O sources and a new control domain to a running system and measure the required code changes. Fail-passive behaviour is validated by disconnecting an active source during operation and through high-frequency stress tests. Cross-layer consistency is verified by checking the homomorphism property on randomly generated effect list pairs. The specific validation procedures and test parameters are reported alongside their results in Sections 6 and 7.

4.4 Experimental Platform

TwinCAT 3 (Beckhoff) is used as the PLC runtime. It supports IEC 61131-3 Structured Text and provides the ADS protocol for communication with external systems. Python is chosen for the middleware because of its ecosystem support for LLM services, IoT protocols, web frameworks, and testing libraries. The architecture does not depend on Python specifically. This choice makes it easier for readers to replicate the system and add new M+O sources of their own. The LLM component runs on Ollama with a local language model. The base configuration uses two M+O input sources: HMI buttons and an LLM agent. An MQTT source is added as part of the horizontal extensibility experiment.

4.5 Implementation Constraints

IEC 61131-3 has a limited type system. Structured Text has no generics and no higher-order functions. The Monoid interface is implemented as a convention: each Monoid provides a combine function and an identity function with matching signatures. This convention is validated by automated testing rather than enforced by the type system. The source code will be made available upon publication.

Section 5: MonoPLC Architecture

5.1 Overview

MonoPLC consists of three layers (Fig. 1). **Layer 1 (PLC)** runs the real-time control logic in IEC 61131-3 Structured Text; all I/O is represented as Effect Monoid elements. **Layer 2 (Bridge Server)** reads effects from the PLC, processes them into a state dictionary exposed via REST API, and writes M+O inputs to the PLC input queue after whitelist validation. **Layer 3 (M+O sources)** contains input sources (HMI buttons, LLM agent, MQTT devices) that produce effects in the same format and connect through the REST API. Data flows in both directions: effects from PLC to middleware for state reconstruction, and effects from M+O sources to PLC for control input.

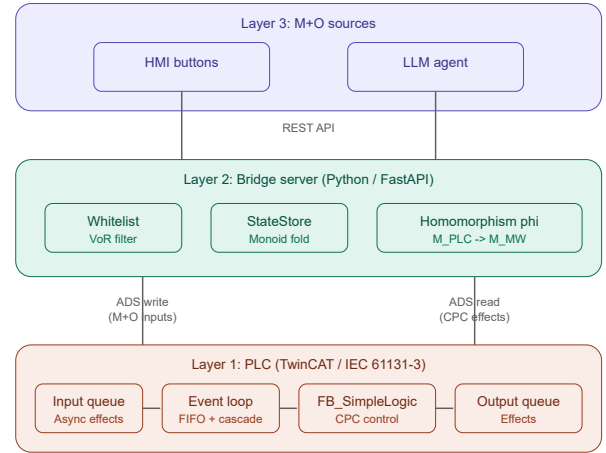


Fig. 1. MonoPLC three-layer architecture: PLC (Layer 1), Bridge Server (Layer 2), M+O sources (Layer 3). All sources produce the same Effect data type; data flows bidirectionally through the Bridge Server.

5.2 PLC-Side Effect Monoid

5.2.1 Effect Data Type

`DUT_Effect` is the atomic element of the system. It represents a single effect. `ENUM_Effect_Type` defines 11 effect types covering I/O commands, publications, alarms, setpoint changes, and system events. Types 3 and 4 (`VALVE_CTRL` , `ALARM`) can only be generated by the PLC control logic; M+O sources are not allowed to produce them (enforced by the whitelist in Section 5.6).

Listing 1. Effect data type (Structured Text).

```
TYPE DUT_Effect :
STRUCT
    EType    : ENUM_Effect_Type; (* 11 types:
    NONE, IOT_PUB, ..., LLM_DECISION *)
    Target   : STRING[22];        (* valve
    name, topic, parameter *)
    Payload  : STRING[64];
    Value    : REAL;
```

```
END_STRUCT
END_TYPE
```

This structure is the same for all effects regardless of source. `EType` determines the semantic category, `Target` identifies what the effect acts on, and `Value` and `Payload` carry numeric and string data.

5.2.2 Effect Monoid

`DUT_Effect_Monoid` is the Monoid carrier: a `STRUCT` containing an `INT` count and an `ARRAY[1..20]` OF `DUT_Effect`. The binary operation `FC_CombineEffects` copies A into the result, then appends elements of B up to the capacity limit $N = 20$; elements beyond N are discarded. The identity element `FC_EmptyEffect` returns an empty array (`Count = 0`). Structured Text initialises all struct fields to zero by default.

5.2.3 Climate Product Monoid

The system monitors two domains: temperature and humidity. Each domain has its own Monoid. The temperature domain uses the Effect Monoid described above — ($\text{Effect}^{\leq 20}$, ++, []) with truncation at capacity $N = 20$. The humidity Monoid (`DUT_Humidity_Monoid`) is a `STRUCT` with a single `REAL` field (`SprayAmount`); `FC_CombineHumidity` adds the two values, and the identity is 0.0 — a standard $(\mathbb{R}, +, 0)$ Monoid. The Climate Product Monoid (`DUT_Climate_Monoid`) is a `STRUCT` containing a `DUT_Effect_Monoid` for temperature and a `DUT_Humidity_Monoid` for humidity. The combine operation `FC_CombineClimate` applies each component's combine independently: it calls `FC_CombineEffects` on the temperature fields and `FC_CombineHumidity` on the humidity fields, returning a new `DUT_Climate_Monoid` with both results. This is the Product Monoid described in Section 3.2. Adding a new domain (e.g. pressure monitoring) means adding a new component to this structure. The existing components do not change.

All domain data exits the PLC through the same output queue as `DUT_Effect` elements; the middleware reconstructs per-domain state by processing this single effect stream with domain-specific mapping functions (Section 5.4).

5.3 Event Processing and Function Blocks

Each scan cycle, the PLC collects M+O inputs and internal events, passes them to function blocks, and routes the resulting effects to an output queue for middleware consumption.

Each function block receives a single `DUT_Effect` and returns a `DUT_Effect_Monoid` (up to 20 effects). Every event is passed to all function blocks; each responds only to the event types relevant to its domain and returns the identity element for irrelevant events. Each function block produces output in its own Monoid: `FB_SimpleLogic` returns a `DUT_Effect_Monoid` (temperature effects), and `FB_HumidityLogic` returns a `DUT_Humidity_Monoid` (humidity delta). These per-domain outputs form a `DUT_Climate_Monoid`, which is then combined independently per domain using `FC_CombineClimate` — the Product

Monoid's combine operation. Effects that require external action (`EFF_VALVE_CTRL`, `EFF_IOT_PUB`) are routed to the output queue; other effects (e.g. setpoint changes, mode commands) are consumed internally by the function blocks. The middleware reads the output queue for state reconstruction (Section 5.4).

`FB_SimpleLogic` is the temperature control block; `FB_HumidityLogic` is the humidity control block. Each maintains its own internal state and responds only to events relevant to its domain. `FB_SimpleLogic` implements hysteresis-based cooling valve control: it maintains `System_En`, `Is_Cooling`, `TempHighLimit`, `TempLowLimit` and applies range validation (e.g. `TempHighLimit` clamped to $[50, 100]^{\circ}\text{C}$) to prevent M+O sources from setting unsafe values. `FB_HumidityLogic` accumulates spray amounts from humidity-related events into a running total using the Humidity Monoid $(\mathbb{R}, +, 0)$. Neither block reads sensors or writes to actuators directly — all I/O is mediated through the effect system.

Listing 2. Effect composition pattern in function blocks (simplified).

```
(* FB_SimpleLogic - temperature control *)
Effects_Out := FC_EmptyEffect();
(* start from identity *)
IF Input_Event.EType = EFF_IOT_CMD_STOP THEN
    Eff_Valve :=
FC_ValveEffect('CoolingValve', FALSE);
    Eff_Network :=
FC_IoTEffect('system/status', ...);
    Effects_Out :=
FC_CombineEffects(Eff_Valve, Eff_Network);
ELSIF ...
(* other branches follow same pattern *)
END_IF;

(* FB_HumidityLogic - humidity accumulation *)
Hum_Out.SprayAmount := 0.0;
(* start from identity *)
IF Input_Event.EType = EFF_SYSTEM_TICK AND
Spray_Active THEN
    Hum_Out.SprayAmount := SprayRate *
CycleTime;
END_IF;
```

Both blocks start from their respective identity elements and produce output only when a relevant event is received. The Product Monoid combine (`FC_CombineClimate`) then merges their outputs: `FC_CombineEffects` concatenates the temperature effects, and `FC_CombineHumidity` adds the humidity values. The addition that accumulates spray amounts happens here, not inside the function block — the FB only produces a per-cycle delta. If no condition is met, the output remains the identity element, and the combine has no effect.

Two distinct algebraic structures operate at different layers. Inside the PLC, the Monoid combine operates on the outputs of function blocks within a scan cycle. Across the PLC-middleware boundary, the output queue accumulates effects over multiple scan cycles into a growing list; this list is consumed by the middleware for state reconstruction (Section 5.4) and cross-layer consistency verification (Section 5.5).

5.4 Middleware-Side Monoid System

Effects that reach the output queue are individual `DUT_Effect` elements. The middleware reconstructs its own view of the system by processing this flat effect stream sequentially.

The middleware defines two Monoids that the homomorphism φ connects:

- **Effect list Monoid** (`list[Effect]`, `++`, `[]`): unbounded list concatenation. This represents the accumulated history of all effects read from the PLC output queue via ADS — unlike the PLC-side Effect Monoid, which is bounded to 20 elements per scan cycle.
- **State dictionary Monoid** (`dict`, `mergeR`, `{}`): right-biased dictionary merge with identity `{}`. When two entries share the same key, the later one wins.

The details of φ and its verification are described in Section 5.5.

The middleware also uses a Product Monoid to track multiple dimensions simultaneously. These dimensions are not limited to the PLC-side control domains; they also include cross-domain metrics such as counters and totals. Each monitoring dimension is registered as a named component with its own Monoid and mapping function. The base configuration includes four components: state (using the State dictionary Monoid), effect count and alarm count (each using $(\mathbb{Z}, +, 0)$), and total spray amount (using $(\mathbb{R}, +, 0)$). For example, the `total_spray_ml` component uses $(\mathbb{R}, +, 0)$ with a mapping function that extracts the `value` field from `EFF_VALVE_CTRL` effects targeting `Humidifier` and returns 0.0 (the identity) for all other effects. Each component sees every effect but responds only to those relevant to its domain — the same broadcast-and-filter pattern used by the PLC-side function blocks (Section 5.3). Adding a new dimension requires one component registration. The number of middleware components is independent of the number of PLC-side control domains — this decoupling is a direct consequence of the Product Monoid structure, which allows each side to define as many or as few components as needed.

The `StateStore` processes incoming effects from the PLC output queue into the Product Monoid. Every effect is broadcast to all components simultaneously; each mapping function extracts its relevant data or returns the identity. The state at any point is a deterministic function of the effect history:

$$\text{state}(t) = \bigoplus_{i=0}^t \text{map}(\text{effect}_i)$$

where $\oplus$ is the Product Monoid combine and `map` maps a single effect to all components.

5.5 Cross-Layer Homomorphism

Cross-layer consistency is typically addressed by defining a shared data format. This guarantees that data fits an expected shape, but not that two sides produce the same result when combining the same data. The PLC-side output queue accumulates effects over multiple scan cycles into a growing list; the middleware processes this list into a state dictionary. These are two different Monoids connected by a homomorphism φ (Section 3.3). The homomorphism guarantees that the two

sides combine data consistently, regardless of how effects are split across transfer cycles.

5.5.1 The Mapping Function φ

φ maps PLC-side effect lists to middleware-side state dictionaries:

```

 $\varphi : M_{\text{PLC}} \rightarrow M_{\text{MW}}$ 

 $\varphi : (\text{list}[\text{Effect}], ++, []) \rightarrow (\text{dict}, \text{merge}_R, \{\})$ 

function  $\varphi(\text{effects} : \text{list}[\text{Effect}]) \rightarrow \text{dict}$ 
  result  $\leftarrow \{\}$ 
  for each  $e \in \text{effects}$  do
    if  $e.\text{type} \in \{\text{TICK}, \text{NONE}\}$  then skip
     $k \leftarrow (e.\text{type}, e.\text{target})$ 
     $\text{result}[k] \leftarrow (e.\text{value}, e.\text{payload})$ 
  return result

```

For any key k that appears more than once, the last assignment wins (last-writer-wins). The FIFO processing order determines which effect is "last": effects are processed in arrival order, and when two effects target the same key, the later arrival overwrites the earlier one. The homomorphism property is proved in Section 6.4 and confirmed by property-based testing.

5.5.2 Generic Homomorphism Framework

A generic `MonoidHomomorphism` class encapsulates two Monoids and the mapping function φ . Its `verify(a, b)` method takes two PLC-side effect lists and checks the homomorphism property by computing both paths — $\varphi(\text{combine}(a, b))$ and $\text{combine}(\varphi(a), \varphi(b))$ — and comparing the results (Fig. 2). In automated testing (Section 4.3), a and b are randomly generated effect lists.

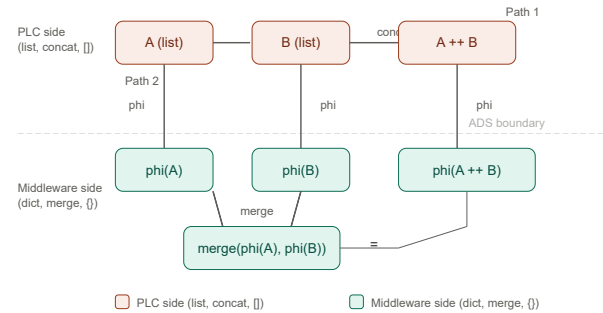


Fig. 2. Monoid Homomorphism verification: two computation paths (combine-then-map vs. map-then-combine) must produce identical results.

5.6 Whitelist and LLM Closed Loop

M+O sources are restricted to 7 of the 11 effect types; safety-critical types (`VALVE_CTRL`, `ALARM`) can only be produced by PLC control logic. This whitelist realises the VoR principle — verifying external requests before they enter CPC — as an effect-type-based restriction on the Effect Monoid.

The LLM agent reads system state, makes a decision, and produces an effect that passes through the whitelist before entering the PLC input queue. This forms a closed loop — state

→ LLM → effect → whitelist → PLC → new effects → process
→ updated state — in which the LLM is one M+O source
among many, not a special case.

5.7 Design Space: Alternative Monoid Implementations

The Monoid interface — a combine operation and an identity element satisfying the axioms — can be implemented by different data structures with different trade-offs. The event loop, function blocks, and cross-layer homomorphism depend only on the interface. Switching implementations requires changing only the combine and identity functions.

Design A: Bounded Concatenation (used in the case study). The combine operation is $A \oplus B = \text{trunc}(N, A ++ B)$ with $N = 20$, where $\text{trunc}(N, xs)$ keeps the first N elements of a list and discards the rest. Associativity under truncation is proved in Section 6.1. Overflow results in data loss (not the identity element), so the fail-passive guarantee (RQ-c) applies to source disconnection, not buffer overflow. Overflow can be tracked through a diagnostic side-channel without modifying the algebraic operation; replacing the overflowed result with ε would break associativity.

Design B: Key-Aware LWW Merge. Deduplicates effects by composite key ($EType, Target$), keeping only the most recent value. Associativity holds because right-biased merge over a key space is associative. The capacity is bounded by the number of distinct key pairs. Suitable where only the latest value matters (setpoints, mode commands); inappropriate where every occurrence matters (alarm counts).

Design C: Priority-Aware Truncation (proposed). Replaces FIFO truncation with priority-sorted truncation, retaining the N highest-priority effects during overflow. Associativity is preserved under stable sorting with deterministic tie-breaking (details in Supplementary Material S3).

Design D: Budget-Limited Processing (proposed). Limits event processing to at most K events per scan cycle without changing the Monoid; unprocessed events carry over to subsequent cycles (details in Supplementary Material S3).

Design	Combine Strategy	Overflow Behaviour	Suitable For
A: Bounded Concat	Array concat, $\text{trunc}(N)$	Truncation (detected)	Low-rate, general use
B: LWW Merge	Key-aware dedup	Natural bound	Setpoint / command effects
C: Priority Truncation (proposed)	Sorted concat, $\text{top}(N)$	Low-priority effects discarded	Safety-critical overflow handling
D: Budget-Limited (proposed)	Same as A/B/C, $\leq K$ per cycle	Delayed processing	Shared scan cycle budget

Designs A and B are implemented in both the middleware and PLC-side Structured Text and validated through property-based testing. Designs C and D are proposed alternatives. The design space is not closed: any combine operation that satisfies closure, associativity, and identity can be substituted without modifying the event loop, the function blocks, or the cross-layer homomorphism.

This section addresses each research question through mathematical analysis and automated testing results.

6.1 RQ-a: Feasibility

Question: Can the Monoid axioms be implemented within IEC 61131-3 and achieve complete separation of CPC from M+O?

Associativity

We show that `FC_CombineEffects` satisfies associativity. Let $A = (a_1, \dots, a_m)$, $B = (b_1, \dots, b_n)$, $C = (c_1, \dots, c_p)$ be three effect arrays.

$(A \oplus B) \oplus C$: First concatenate A and B to get $(a_1, \dots, a_m, b_1, \dots, b_n)$, then concatenate with C to get $(a_1, \dots, a_m, b_1, \dots, b_n, c_1, \dots, c_p)$.

$A \oplus (B \oplus C)$: First concatenate B and C to get $(b_1, \dots, b_n, c_1, \dots, c_p)$, then concatenate with A to get $(a_1, \dots, a_m, b_1, \dots, b_n, c_1, \dots, c_p)$.

Both results are identical. Associativity holds.

Bounded array. The implementation uses a fixed-size array of 20 elements. When the total exceeds 20, elements are truncated from the right. We show that associativity still holds under truncation. The operation is equivalent to $A \oplus B = \text{trunc}(20, A ++ B)$. Two key properties are needed:

$$\begin{aligned} \text{trunc}(N, \text{trunc}(N, xs) ++ ys) &= \text{trunc}(N, xs ++ ys) \quad (\text{left}) \\ \text{trunc}(N, xs ++ \text{trunc}(N, ys)) &= \text{trunc}(N, xs ++ ys) \quad (\text{right}) \end{aligned}$$

For the left property: if $|xs| \geq N$, then $\text{trunc}(N, xs)$ already has N elements, so appending ys and truncating again gives the same first N elements as $\text{trunc}(N, xs ++ ys)$. If $|xs| < N$, then $\text{trunc}(N, xs) = xs$, so the left side reduces to the right side directly. For the right property: the elements removed from ys by truncation sit at positions $\geq |xs| + N \geq N$ in the concatenation, so they fall outside the first N elements of the result regardless of $|xs|$. Applying the left property to $(A \oplus B) \oplus C$ and the right property to $A \oplus (B \oplus C)$, both reduce to $\text{trunc}(20, A ++ B ++ C)$, and associativity holds even at capacity.

The identity element (empty array `[]`) also holds under truncation: $\text{trunc}(N, [] ++ A) = \text{trunc}(N, A) = A$ whenever $|A| \leq N$, which is always the case because the output of every combine is itself truncated to at most N elements. The symmetric case $\text{trunc}(N, A ++ []) = A$ follows by the same argument.

Truncation means that effects may be lost when the buffer is full. In practice, a single scan cycle produces far fewer than 20 effects.

Product Monoid

The Climate Product Monoid `DUT_Climate_Monoid` combines the Effect Monoid and the Humidity Monoid independently per domain. By the Product Monoid theorem (Section 3.2), closure, associativity, and identity are inherited from the components. No separate proof is needed.

Section 6: Formal Analysis

Automated Testing

We tested each Monoid type using property-based testing: list concatenation ($++$), right-biased dictionary merge (merge_R), and integer addition ($+$). For each, we checked associativity ($(A \oplus B) \oplus C = A \oplus (B \oplus C)$) with 100 random triples, left identity $\varepsilon \oplus A = A$ with 50 random cases, and right identity $A \oplus \varepsilon = A$ with 50 random cases. All 600 cases (3 Monoids $\times$ 200 cases each) passed.

Answer to RQ-a: The Monoid axioms can be implemented within IEC 61131-3 Structured Text constraints. Associativity holds even with bounded arrays. The Product Monoid extends to multiple domains. The function blocks achieve complete separation of CPC from M+O (Section 5.3).

6.2 RQ-b: Extensibility

Question: Can new input sources be added without modifying core control logic (horizontal extensibility), and can new monitoring domains be added while preserving all existing control logic (vertical extensibility)?

Adding a New Input Source

All M+O input sources connect to the system through the same mechanism: produce an `Effect`, push it to `Async_Input_Queue`. The PLC event loop processes effects from this queue without checking their origin. `FB_SimpleLogic` receives an event and responds based on its `EType`, not on where it came from.

To add a new M+O source:

1. The source produces an `Effect` with an appropriate `EType`, `Target`, `Value`, and `Payload`.
2. It pushes the effect to the input queue via the middleware REST API.

No change is needed in `FB_SimpleLogic`, `Control_LOOP`, or any PLC-side code. The Closure axiom enforces this: all effects must be elements of the same Monoid, so all sources must produce the same data type. Fig. 3 illustrates how a new source feeds into the unchanged Monoid structure.

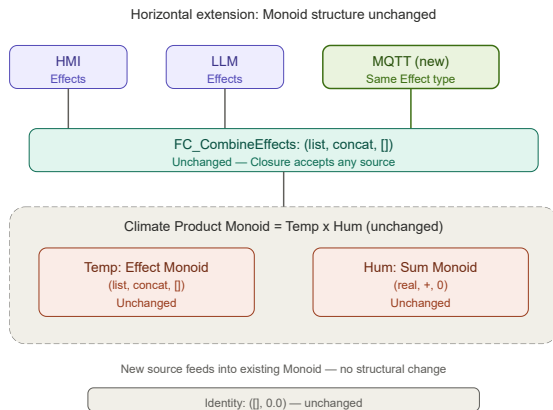


Fig. 3. Horizontal extension: a new M+O source feeds effects into the unchanged Monoid composition via the Closure axiom.

Adding a New Monitoring Dimension

On the middleware side, adding a new monitoring dimension requires registering a single `MonoidComponent` (e.g. `peak_temp` using `MaxMonoid`). The `DictProductMonoid` applies this component automatically during processing. The state reconstruction, the REST API, and the LLM state prompt all include the new dimension without code changes. This follows from the Product Monoid theorem: a new component inherits Monoid properties automatically.

Adding a New Control Domain (Vertical Extensibility)

The two cases above are horizontal extensions: a new source or a new monitoring dimension on the middleware side. A stronger test is vertical extensibility: adding a complete new control domain on the PLC side.

Adding a pressure monitoring domain requires new PLC components (data type, combine function, control function block) and an extension of the Product Monoid structure. The key property is that existing control logic is not modified. `FB_SimpleLogic` and `FB_HumidityLogic` remain unchanged because the Product Monoid composes domains independently. Each function block receives the same event stream but responds only to events relevant to its domain. The Product Monoid theorem guarantees that adding a new component does not affect the behaviour of existing components.

The modifications are limited to framework-level wiring: extending the Product Monoid type by one field, adding one combine call, and instantiating the new function block in the event loop. These are additive changes. No existing logic is modified or deleted. Fig. 4 shows how the Product Monoid is extended with a new component while existing components remain unchanged.

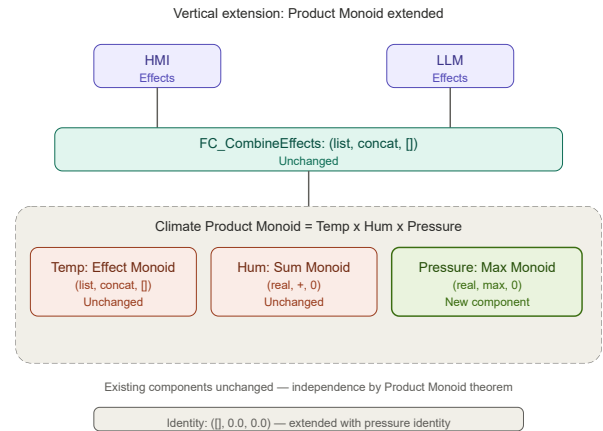


Fig. 4. Vertical extension: a new Pressure component is added to the Product Monoid; existing Temperature and Humidity components remain unchanged.

Whitelist as VoR Implementation

The VoR principle requires that M+O commands be validated before they enter CPC. MonoPLC implements this as an effect-type-based restriction on the Effect Monoid: a configurable whitelist of 7 allowed effect types out of 11. Effects whose types fall outside this set are rejected before entering the system. The

allowed effects form a subset that is closed under the Monoid operation — combining two allowed effects always produces an allowed effect — so the whitelist is compositionally consistent. It can be reconfigured without modifying PLC code.

Answer to RQ-b: New M+O sources are added by producing effects in the standard format (horizontal extensibility). New monitoring dimensions are added by registering a Product Monoid component. New control domains are added by extending the Product Monoid structure with additive code changes, while existing control logic remains unchanged (vertical extensibility). The whitelist provides an effect-type-based implementation of the VoR concept.

6.3 RQ-c: Fail-Passive Behaviour

Question: Does the Identity axiom, combined with the design decision to represent absent sources as the identity element, guarantee that the system operates correctly when input sources fail, disconnect, or produce no output?

The Monoid axioms require an identity element ε such that $\varepsilon \oplus a = a$ for all a . In MonoPLC, ε is `FC_EmptyEffect()`: an effect array with `Count = 0`. The design decision is to represent an absent or disconnected source as contributing ε rather than an error signal or a fallback value. Given this decision, the identity law guarantees that the absent source does not affect any composition — at every point where effects are combined — without per-source conditional logic. This compositional property distinguishes the Monoid approach from conventional null handling, where encountering null at a composition point requires a conditional check and missing such a check at any single point leads to undefined behaviour. When an M+O source disconnects, it stops producing effects, and for any current state s : $\text{act}(s, \varepsilon) = s$. The PLC control logic continues its normal operation and the system remains predictable. We confirmed this by disconnecting the LLM agent during operation; the PLC continued to process system tick events and HMI inputs without error conditions.

Answer to RQ-c: The Identity axiom, combined with the design decision to represent absent sources as ε , provides a compositional fail-passive guarantee at every composition point without per-source conditional logic. Buffer overflow is a separate concern addressed by the choice of Monoid implementation (Section 5.7) and discussed in Section 8.4.

6.4 RQ-d: Cross-Layer Consistency

Question: Can the Monoid Homomorphism provide verifiable cross-layer consistency between the PLC and the middleware through a self-verifying check?

Homomorphism Proof

The homomorphism φ operates at the cross-layer boundary between the output queue and the middleware. Its domain is the unbounded effect list accumulated in the output queue across multiple scan cycles — not the bounded array used by `FC_CombineEffects` within a single scan cycle. The PLC-side Monoid for the homomorphism is therefore $(\text{list}[\text{Effect}], ++, [])$ without truncation. The bounded array and its truncation semantics (Section 5.7) are internal to the PLC's scan cycle processing; effects that reach the output queue

are individual `DUT_Effect` elements, and the output queue itself is an unbounded list.

We show that $\varphi(A \oplus_1 B) = \varphi(A) \oplus_2 \varphi(B)$ where $\oplus_1$ is list concatenation and $\oplus_2$ is right-biased dictionary merge.

Let $A = [a_1, \dots, a_m]$ and $B = [b_1, \dots, b_n]$.

Left side: $\varphi(A \oplus_1 B) = \varphi([a_1, \dots, a_m, b_1, \dots, b_n])$. The function iterates through the concatenated list, generating key-value pairs. For any key k , the last occurrence determines the value (last-writer-wins).

Right side: $\varphi(A) \oplus_2 \varphi(B) = \text{merge}_R(\varphi(A), \varphi(B))$. Each list is mapped independently, then the two dictionaries are merged with right-biased semantics. For any key k present in both, $\varphi(B)$'s value overwrites $\varphi(A)$'s value.

For any key k , three cases arise:

- k appears only in A : both sides produce the same value from A .
- k appears only in B : both sides produce the same value from B .
- k appears in both: the left side takes the value from B (because B elements come after A in the concatenated list). The right side also takes the value from $\varphi(B)$ (because dictionary merge overwrites with the second operand). Both sides agree.

Therefore $\varphi(A \oplus_1 B) = \varphi(A) \oplus_2 \varphi(B)$.

Identity preservation: $\varphi([]) = \{\}$. The empty list produces an empty dictionary, which is the identity of the middleware-side Monoid.

Noise filtering: The filtering of `SYSTEM_TICK` and `NONE` effects does not affect the result because filtering is itself a homomorphism on lists ($\text{filter}(A++B) = \text{filter}(A)++\text{filter}(B)$), and the composition of two homomorphisms is a homomorphism.

Automated Testing

We generated 200 random pairs of effect lists. For each pair (A, B) :

1. We computed $\varphi(A \oplus_1 B)$ (combine on PLC side, then map).
2. We computed $\varphi(A) \oplus_2 \varphi(B)$ (map each side, then combine on middleware side).
3. We checked that results 1 and 2 are identical.

The test cases included:

- Effect lists with overlapping keys (same `EType` and `Target`).
- Effect lists containing noise signals (`SYSTEM_TICK`, `NONE`).
- Empty effect lists (testing identity preservation: $\varphi([]) = \{\}$).

All 200 cases passed. The `MonoidHomomorphism.verify()` method was used for each check.

Answer to RQ-d: The Monoid Homomorphism preserves composition structure across the PLC-middleware boundary. The homomorphism check is self-verifying: it compares two computation paths on the same input without requiring manually defined expected outputs.

Section 7: Case Study

7.1 Scenario Description

The case study uses a temperature and humidity control scenario. The PLC controls a cooling valve based on temperature readings using hysteresis logic. A humidity control module tracks spray amounts. The base configuration connects two M+O input sources:

- **HMI buttons:** Local buttons for system start, stop, and reset. These produce `EFF_IOT_CMD_START`, `EFF_IOT_CMD_STOP`, and `EFF_IOT_CMD_RESET` effects.
- **LLM agent:** A local language model (Ollama) that reads system state, analyses conditions, and produces effects such as `EFF_SETPOINT_CHANGE` or `EFF_IOT_CMD_STOP`.

A third source (MQTT device) is added as part of the horizontal extensibility experiment (Section 7.2, RQ-b). It produces `EFF_SETPOINT_CHANGE` effects through the same REST API. A pressure monitoring domain is added as part of the vertical extensibility experiment (Section 7.2, RQ-b), introducing a new data type, combine function, and control function block on the PLC side.

All sources connect through the same mechanism described in Section 5: wrap as Effect, push to input queue.

7.2 RQ Validation

RQ-a: Feasibility

The Monoid law tests (Section 6.1) all passed. On the PLC side, the Climate Product Monoid was tested via ADS connection to a running TwinCAT instance, confirming component independence (humidity changes do not affect temperature outputs) and correct accumulation.

RQ-b: Extensibility

We added a new M+O input source (an MQTT device sending `EFF_SETPOINT_CHANGE` to adjust `TempHighLimit`) to the running system. The following changes were required:

- `FB_SimpleLogic` : **0 lines modified.**
- `Control_LOOP` : **0 lines modified.**
- Other PLC code: **0 lines modified.**
- `plc_bridge.py`, `state_store.py`, `monoid.py`, `models.py` : **0 lines modified.**
- New file: approximately 30 lines (MQTT source handler that produces `Effect` objects and pushes them via the REST API).

The PLC correctly processed the new source's effects. The StateStore included the new effect (`EFF_SETPOINT_CHANGE.TempHighLimit: {value: 78.0, payload: "MQTT_Sensor_001"}`). The LLM state prompt automatically included the new data through the generic `phi` mapping.

We also added a new monitoring dimension (`peak_temp`) to the middleware-side Product Monoid by adding one line to `PRODUCT_COMPONENTS`. The state reconstruction, REST API, and LLM state prompt included the new dimension without further changes. The `peak_temp` value was correctly computed as $\max(70.0, 85.0, 95.0) = 95.0$. The architectural impact of horizontal extension is shown in Fig. 5: Layer 1 and Layer 2 remain unchanged.

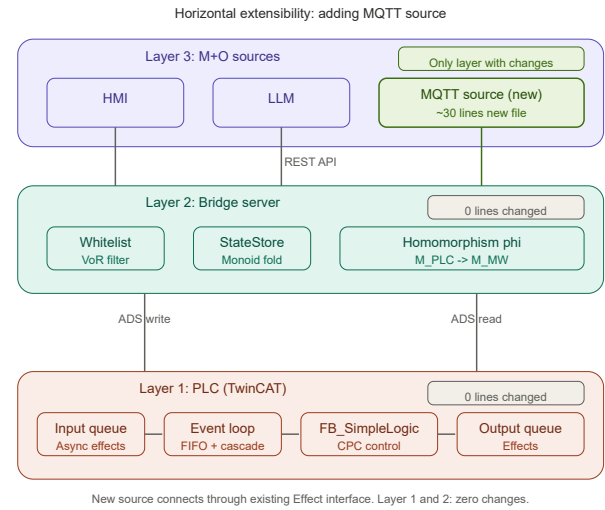


Fig. 5. Horizontal extensibility case study: MQTT source added with zero code changes to Layer 1 (PLC) and Layer 2 (Bridge Server).

To test vertical extensibility, we added a complete pressure monitoring domain to the system. This required new PLC-side components: a data type (`DUT_Pressure_Monoid`, using $(\mathbb{R}, \max, 0)$), a combine function (`FC_CombinePressure`), and a control function block (`FB_PressureLogic` with hysteresis control for a vent valve). The Climate Product Monoid was extended by adding one field and one combine call.

`Control_LOOP` was extended by 8 lines to instantiate and wire the new function block. On the middleware side, 6 lines were added to register a `peak_pressure` component using `MaxMonoid` (map function, import, and component registration). The existing control logic required zero modification: `FB_SimpleLogic` (temperature) and `FB_HumidityLogic` (humidity) were unchanged. All 53 existing tests passed without modification. Component independence was confirmed: pressure changes did not affect temperature outputs or humidity accumulation. The Product Monoid theorem guarantees this independence, and the test results confirm that the implementation matches the theorem. Fig. 6 shows the resulting architecture with the new pressure domain added.

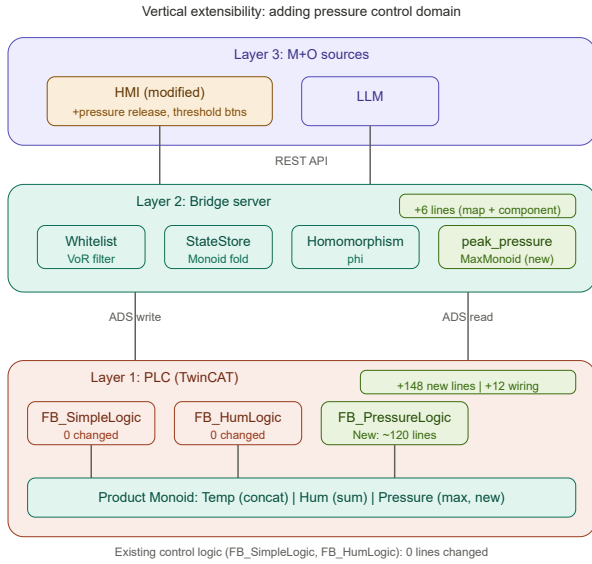


Fig. 6. Vertical extensibility case study: pressure domain added on both PLC and middleware sides; existing function blocks unchanged, all 53 prior tests pass.

RQ-c: Fail-Passive Behaviour

We disconnected the LLM agent while the system was running. The PLC continued to process system tick events and HMI inputs. Temperature control continued normally. No error conditions were triggered. The system remained predictable.

We also ran a high-frequency disturbance stress test to verify that rapid M+O inputs cannot compromise CPC stability. In the first test, 200 mixed effects (setpoint changes, start/stop/reset commands) were injected through the REST API. The PLC processed all 200 effects while maintaining temperature in the normal operating range (66–83°C). The input queue depth remained at 0 throughout — the PLC’s 10 ms scan cycle consumed each effect before the next injection completed. In the second test, 50 alternating start/stop commands were sent at 50 commands per second. The PLC processed each command in FIFO order and settled to a valid final state. In the third test, 100 random setpoint changes (TempHighLimit in the range 60–95°C) were sent in 5 seconds. The system correctly applied last-writer-wins semantics and settled to the most recent value. Across all three stress tests, the PLC control logic was unaffected by the disturbances.

RQ-d: Cross-Layer Consistency

The homomorphism tests described in Section 6.4 all passed (200 random pairs). An additional 100 pairs were verified through the `MonoidHomomorphism.verify()` bridge class and 100 through the `transport_combine` equivalence check, all producing identical results. Identity preservation ($\varphi([]) = \{ \}$) was confirmed.

7.3 Quantitative Metrics

The system was run for 30 seconds with the PLC scan cycle configured at 10 ms.

Effect Distribution. During the 30-second measurement, 28 effects were collected from the PLC output queue. The

distribution was: `EFF_IOT_PUB` 24 (85.7%) and `EFF_VALVE_CTRL` 4 (14.3%). The dominance of `IOT_PUB` effects reflects the system’s heartbeat mechanism, which periodically publishes status updates.

Whitelist. All 11 effect types were tested against the whitelist. The 4 blocked types (`VALVE_CTRL`, `ALARM`, `SYSTEM_TICK`, `LLM_DECISION`) were correctly rejected. The 7 allowed types were correctly accepted. No type was both allowed and blocked.

Scan Cycle Execution Time. We recorded the actual task execution time for each scan cycle using TwinCAT TE1300 Scope View during the stress test (13,705 cycles over 137 seconds). The results are summarised below.

Metric	Value
Mean	44.7 μ s
Median	44.6 μ s
Std. dev.	16.1 μ s
Min	18.5 μ s
p95	62.6 μ s
p99	87.1 μ s
p99.9	154.8 μ s
Max	1118.3 μ s
Configured cycle time	10,000 μ s
Mean utilisation	0.45%
Worst-case utilisation	11.18%

The execution time was below 100 μ s in 99.5% of all cycles. The single highest spike (1118.3 μ s) occurred during a cycle that processed only one event. Even this worst case consumed only 11.2% of the 10 ms scan cycle budget. During 89.6% of cycles, only one event (the system tick) was processed; the maximum observed was 8 events per cycle. The event bus queue depth remained at 0 throughout — the PLC consumed every input event within the same scan cycle in which it arrived.

Overflow Counter. An overflow counter was added to the event bus input transfer. The counter remained at zero throughout all experiments, including the stress test with 200 injected effects. The PLC consumed each input effect within one scan cycle.

Test Suite Summary. The full test suite comprised 64 tests across 8 test files, including a high-frequency stress test with 200 injected effects and a vertical extensibility test adding a complete pressure monitoring domain. All tests passed with zero failures. The 53 pre-existing tests passed unchanged after the pressure domain was added (regression). Hypothesis generated approximately 1300 random examples across all property-based tests.

Section 8: Discussion

8.1 Summary of Results

The case study and formal analysis address all four research questions. The Monoid axioms can be satisfied within IEC 61131-3 (RQ-a). New sources can be added without modifying core control logic, and new monitoring domains can be added while preserving all existing function blocks (RQ-b). The Identity element, combined with the design decision to map absent sources to identity, provides compositional fail-passive

behaviour when sources disconnect (RQ-c). The Homomorphism preserves cross-layer consistency (RQ-d). Beyond the algebraic properties, the unified Effect type enforced by Closure also yields an engineering benefit: the StateStore automatically includes any new effect type or target in the system state, so new sensors or actuators require no manual wiring for LLM state perception.

8.2 Comparison with Existing Approaches

Dimension	OO Patterns [3,4]	IEC 61499 [5]	AAS [21]	LLM Integration [13-15]	MonoPLC
Structural separation	Yes	Yes	Yes	No	Yes
Algebraic composition laws	Not designed for this	Has event-driven execution semantics (IEC 61499-1); composition is event-based, not algebraic	Not designed for this	Not designed for this	Yes (Monoid axioms)
Fail-passive on source loss	Application-dependent	Application-dependent	Not specified	Not addressed	Yes (Identity + design decision)
Cross-layer consistency	Not addressed	Not addressed	Not addressed	Not addressed	Yes (Homomorphism)
Zero-modification extensibility	Partial (new classes)	Partial (new FBs)	Yes (Submodels)	No	Yes (Closure + Product)
Standardised information model	No	Event/data connections	Yes (IEC 63278-1)	No	Effect type system
VoR implementation	N/A	N/A	N/A	N/A	Effect-type whitelist + range validation

Each approach in this table was designed for a different purpose; where a cell reads "Not designed for this," it means the approach addresses a different concern, not that it is deficient. MonoPLC addresses a specific gap — algebraic composition laws with verifiable properties for M+O input integration — that sits alongside these existing contributions.

8.3 Cross-Layer Consistency Under System Evolution

When the system evolves — whether by changing the PLC-side combine operation, adding a new effect type, or modifying middleware-side combination — the homomorphism provides a guarantee that format validation does not: the developer re-verifies φ against the changed combine operations, and if the two sides have become semantically inconsistent, the property-based test generator finds a counter-example. The verification is local — only φ and the two combine operations need to be re-checked, not the full system pipeline — and self-verifying (no manually defined expected outputs are needed).

Dimension	Conventional Practice	Monoid Homomorphism
What is guaranteed	Structural compatibility + correctness for tested cases	Consistency of composition semantics
Semantic change visibility	Depends on test coverage; format checks pass even when semantics diverge	Detected by re-verifying two algebraic properties of φ
Verification scope after change	Global: re-run tests across the full pipeline	Local: re-check φ against the two combine operations
Re-verification cost	Proportional to test suite size	Constant: two properties, testable with randomly generated inputs

The homomorphism does not replace integration testing. It provides a specific guarantee — that composition semantics are preserved across the communication boundary — that format validation and testing do not systematically cover.

8.4 Limitations and Threats to Validity

Single scenario. The case study uses one scenario (temperature, humidity, and pressure control) with hysteresis-based control logic. Industrial plants typically involve hundreds of control loops, coupled MIMO systems, and advanced control strategies (PID, MPC). The architecture does not constrain the control

algorithm used inside function blocks, but validation is limited to the proof-of-concept scenario presented. We do not claim industrial-scale readiness; we claim that the algebraic architecture is feasible within IEC 61131-3 constraints and that its extensibility properties hold.

Single PLC platform. The implementation uses TwinCAT (Beckhoff). Other IEC 61131-3 platforms (Siemens, Schneider, Omron) have different runtime characteristics. The architecture is defined in standard Structured Text, but portability has not been tested.

Scan cycle budget and worst-case bounds. The event loop processes all events within a single scan cycle. The static worst-case bound is derived from the configured buffer capacities:

Resource	Capacity	Source
Async input queue	100 slots	Ring buffer size (configurable)
Event bus	200 slots	Ring buffer size (configurable)
Effect Monoid array	20 elements	Fixed-size ST array
Cascade depth	1 level	Output effects route to output queue, not back to logic path
Max events per cycle (static bound)	4,300	Compositional bound from above capacities

The static bound (4,300 event-level operations) was never approached during the stress test (maximum 8 events per cycle; measured execution times are reported in Section 7.3). A formal worst-case execution time (WCET) analysis using platform-specific static analysis tools is recommended for safety-critical deployments.

Event bus overflow. The event bus uses a ring buffer with overflow guards; the overflow counter remained at zero in all experiments. Overflow represents data loss, not the absence of input, so the fail-passive guarantee (RQ-c) does not apply to this case. Section 5.7 presents alternative Monoid implementations that address overflow through deduplication (Design B), priority-aware truncation (Design C), and budget-limited processing (Design D).

Setpoint value bounds and security. M+O sources can modify control parameters through `EFF_SETPOINT_CHANGE` effects. The whitelist restricts which effect types are accepted, and range clamping within the function blocks prevents out-of-range values (e.g. `TempHighLimit` clamped to `[50,100]°C`). Together these provide effect-type and value-range input validation. However, the system does not include source authentication, rate limiting, or IEC 62443 threat modelling. A comprehensive security analysis is outside the scope of this paper and is identified as future work.

Section 9: Conclusion and Future Work

9.1 Conclusion

This paper presented MonoPLC, an architecture that adopts Monoid axioms as architectural constraints for integrating M+O inputs into IEC 61131-3 PLC control systems. The architecture was validated through mathematical analysis, property-based testing, and a case study on TwinCAT. The results address all four research questions:

- The Monoid axioms can be implemented in IEC 61131-3 Structured Text, including the Product Monoid for multiple monitoring domains (RQ-a).
- New M+O sources can be added without modifying core control logic (horizontal extensibility), and new monitoring domains can be added while preserving all existing function blocks (vertical extensibility) (RQ-b).
- The Identity axiom, combined with the design decision to represent absent sources as the identity element, provides compositional fail-passive behaviour when M+O sources disconnect — without per-source conditional logic at any composition point (RQ-c).
- The Monoid Homomorphism provides verifiable cross-layer consistency between the PLC and the middleware. The homomorphism check is self-verifying — it compares two computation paths on the same input without requiring manually defined expected outputs (RQ-d).

These properties are not consequences of the axioms alone; they emerge from how the axioms interact with specific design decisions — representing absent sources as the identity element, encoding all I/O as elements of a single Effect type, and connecting layers through a structure-preserving mapping. The algebraic interface also enables multiple Monoid implementations (Section 5.7) with different trade-offs in overflow handling, worst-case execution time, and semantic expressiveness — the architecture remains unchanged across all of them. These guarantees cover the composition of M+O inputs; the internal control logic that processes individual events is deterministic sequential code validated through testing.

9.2 Future Work

Persistent effect log. The current implementation uses a bounded in-memory buffer for effect history. Persisting the full effect log to a database would enable time-travel state reconstruction and parallel fold on the middleware side — both well-known applications of associativity in functional programming and distributed systems [20,21] — without additional architectural changes.

AAS and OPC UA integration. The Effect Monoid and Product Monoid structures could be mapped to AAS Submodels (IEC 63278-1). Each Product Monoid component (state, effect_count, alarm_count) could be exposed as a standardised AAS Submodel, making MonoPLC potentially interoperable with the broader Industry 4.0 ecosystem. The ADS communication layer could be replaced by OPC UA, aligning with the NOA information model and the OPC UA companion specification for AAS (OPC 30270). This would allow MonoPLC's algebraically guaranteed state to be consumed by AAS-compliant systems such as MES, ERP, or digital twin platforms.

Industrial-scale validation. The case study uses a single temperature and humidity control scenario. Applying MonoPLC to larger industrial systems with multiple interacting control loops would test the scalability of the approach.

Alternative Monoid implementations. Section 5.7 presented four designs; Designs A and B are implemented and tested on the middleware side with corresponding PLC-side Structured Text. Evaluating these designs under realistic industrial workloads — including high-frequency event streams, multi-

source contention, and long-running operation — is needed to guide implementation selection for specific deployment scenarios.

References

- [1] M. A. Sehr et al., "Programmable logic controllers in the context of Industry 4.0," *IEEE Trans. Ind. Informat.*, vol. 17, no. 5, pp. 3523–3533, May 2021.
- [2] B. Vogel-Heuser et al., "Challenges for software engineering in automation," *J. Softw. Eng. Appl.*, vol. 7, no. 5, pp. 440–451, 2014.
- [3] E. Faldella, A. Paoli, A. Tilli, M. Sartini, and D. Guidi, "Architectural design patterns for logic control of manufacturing systems: The generalized device," in *Proc. XXII Int. Symp. Information, Communication and Automation Technologies (ICAT)*, Sarajevo, Bosnia and Herzegovina, 2009, pp. 1–7.
- [4] M. Bonfè et al., "Design patterns for model-based automation software design and implementation," *Control Eng. Pract.*, vol. 21, no. 11, pp. 1608–1619, 2013.
- [5] V. Vyatkin, "IEC 61499 as enabler of distributed and intelligent automation: State-of-the-art review," *IEEE Trans. Ind. Informat.*, vol. 7, no. 4, pp. 768–781, Nov. 2011.
- [6] NAMUR, "NE 175: NAMUR Open Architecture," 2020.
- [7] E. Moggi, "Notions of computation and monads," *Inf. Comput.*, vol. 93, no. 1, pp. 55–92, 1991.
- [8] P. Wadler, "Monads for functional programming," in *Advanced Functional Programming*, 1995, pp. 24–52.
- [9] D. Orchard and T. Petricek, "Embedding effect systems in Haskell," in *Proc. ACM SIGPLAN Haskell Symp.*, 2014, pp. 13–24.
- [10] G. Plotkin and J. Power, "Algebraic operations and generic effects," *Appl. Categ. Structures*, vol. 11, no. 1, pp. 69–94, 2003.
- [11] G. Plotkin and M. Pretnar, "Handlers of algebraic effects," in *Proc. 18th European Symp. Programming (ESOP 2009)*, 2009, pp. 80–94.
- [12] D. Bohlender and S. Kowalewski, "Compositional verification of PLC software using horn clauses and mode abstraction," *IFAC-PapersOnLine*, vol. 51, no. 7, pp. 428–433, 2018.
- [13] Y. Xia, M. Shenoy, N. Jazdi, and M. Weyrich, "Towards autonomous system: Flexible modular production system enhanced with large language model agents," in *Proc. IEEE 28th Int. Conf. Emerging Technologies and Factory Automation (ETFA)*, 2023, pp. 1–8.
- [14] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE 30th Int. Conf. Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8.

- [15] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language models," *arXiv preprint* arXiv:2409.18009, 2024.
- [16] E. Rivas and M. Jaskelioff, "Notions of computation as monoids," *J. Funct. Programming*, vol. 27, e21, 2017.
- [17] R. Šusta, "Application of algebraic theory of automata to PLC programs," Ph.D. dissertation, Czech Technical University, Prague, 2003.
- [18] T. A. V. Teatro, "Categories in control systems software: Toward a unified theory of programming and control," Ph.D. dissertation, Ontario Tech University, Oshawa, ON, Canada, 2023.
- [19] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008.
- [20] N. Preguiça, C. Baquero, and M. Shapiro, "Conflict-free replicated data types (CRDTs)," in *Encyclopedia of Big Data Technologies*, S. Sakr and A. Zomaya, Eds. Cham: Springer, 2018.
- [21] Industrial Digital Twin Association (IDTA), "Specification of the Asset Administration Shell — Part 1: Metamodel," IDTA-01001-3-0-1, Jun. 2024. [Online]. Available: <https://industrialdigitaltwin.org/en/content-hub/aasspecifications>
- [22] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Graduate Texts in Mathematics, vol. 5. New York: Springer-Verlag, 1998.
- [23] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. 5th ACM SIGPLAN Int. Conf. Functional Programming (ICFP '00)*, 2000, pp. 268–279.
- [24] T. Hou, M. Arntzenius, and M. Willsey, "Stream programs are monoid homomorphisms with state," *arXiv preprint* arXiv:2507.10799, 2025.
- [25] C. Power, "Algebraic approaches to distributed data systems," Ph.D. dissertation, University of California, Berkeley, 2025.
- [26] D. MacIver, Z. Hatfield-Dodds, and many contributors, "Hypothesis: A Python library for property-based testing," 2025. [Online]. Available: <https://hypothesis.readthedocs.io>